

Elevator Vertical Distance Estimation

Eyal Yakir Oria

April 20, 2026

Abstract

1 Introduction

2 Data Collection

This section documents, end to end, how the dataset underlying this work was gathered, how it is turned into a usable ground truth, and the engineering problems we had to solve along the way. It is written so that a developer who has never touched the repository can reproduce the pipeline from scratch by reading this section alongside `src/data/README.md`.

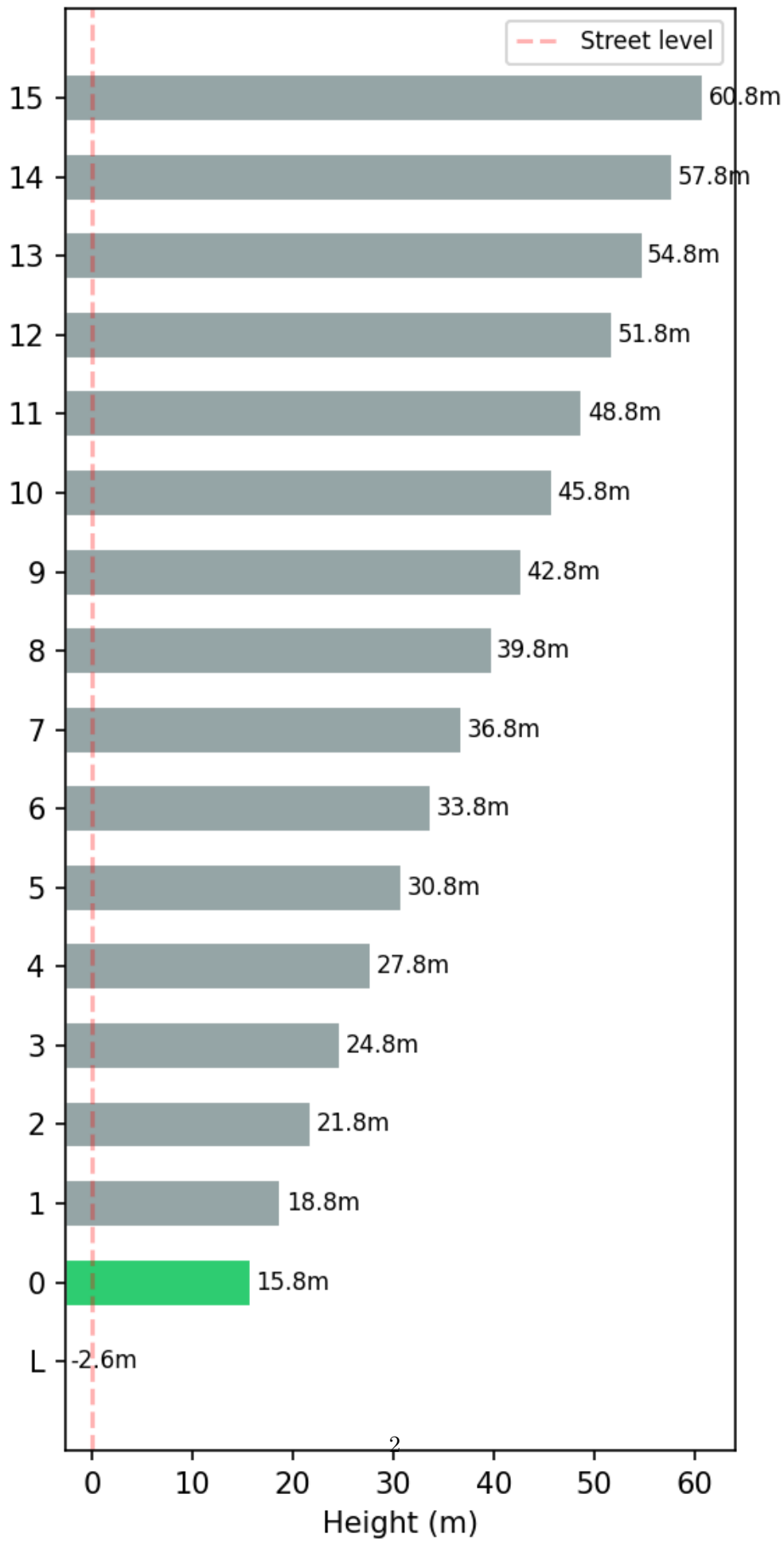
2.1 Experimental Design

We ran a controlled field campaign across **eight different buildings** in central Israel, each with a different elevator, cabin geometry, motor characteristic, and floor-to-floor height. The goal of the campaign was to obtain many statistically-independent elevator rides covering a realistic distribution of buildings (low-rise residential, mid-rise office, hotel, university) so that any algorithm trained or evaluated on this data is not just fitted to a single elevator. The buildings sampled in this campaign are listed in Table 1; Figure 1 summarises their height distribution.

Building (internal name)	Type	Location
milleniumHotel	Hotel	Tel Aviv
acroBuilding	Office	Tel Aviv
beitMansour1	Residential	Ra'anana
beitYitzchakiRaana	Residential	Ra'anana
BarIlana2Herzeliya	Residential	Herzeliya
Haari3	Residential	[TODO: location]
[TODO: building_7]	[TODO: type]	[TODO: location]
[TODO: building_8]	[TODO: type]	[TODO: location]

Table 1: The eight buildings used in the field campaign. Internal names match the `location` field in `rawData/` experiment folders.

Bar-Ilan Building Floor Heights (from Gramushka)



Phones. In every ride we simultaneously recorded data from **four phones**. Using four devices per ride serves two purposes. First, it gives us heterogeneous sensor hardware on a single, perfectly synchronised physical event, which lets us study cross-device variance (different MEMS accelerometers, different sampling jitter, different barometer quality) without confounding it with different rides. Second, it guarantees that at least one of the four devices carries a **barometric pressure sensor** that we can use as a ground-truth source, even when one of the target phones under evaluation does not have one. Typical device mix across the campaign:

- SamsungSM-S911B — Galaxy S23 (has barometer).
- GooglePixel10 / GooglePixel — Pixel family (has barometer).
- Xiaomi22101320I — Redmi Note 12 family (no barometer).
- SamsungSM-A235F — Galaxy A23 (no barometer).
- GalaxyZFlip6 — foldable (has barometer).

The experimenters (eyalyakir, UriyaCohenEliya, RoyTurgeman) carried the devices together in a single hand or bag so that all four phones experience nearly identical motion, vibration, and acoustic environment during a ride.

Folder and file naming. Every recording is stored under `src/data/rawData/<experimenter>_<location>` to make the origin of every sample unambiguous from the path alone. This convention is documented in `src/data/README.md` and is enforced only by convention — the loader (§2.5) does not parse it, but every analysis notebook groups by it.

2.2 Logging: The Barometer

On every phone, a sensor-logging Android application subscribes to every inertial and environmental sensor the device exposes and dumps every sample, verbatim, to a tab-separated text file `sensorLog_<ISO-timestamp>.txt`. One row per sample, no header, format:

```
<timestamp_ms>\t<SENSOR>\t<v1>\t<v2>...
```

The sensors we rely on are summarised in Table 2.

The most important channel for ground truth is the **barometer (PRS)**. The pressure sensor is sampled in parallel with the other sensors and is the sole signal the elevator detector (§2.3) acts on. Because not every device in the fleet has a barometer, we attach a secondary device (typically a Pixel or S23) that *does* have one to every ride whose primary phone is pressure-less; its recording lives in a `forBarometer/` sub-folder under the primary experiment’s directory. This secondary-device trick is central to the time-alignment problem discussed in §2.4.

Tag	Sensor	Columns
ACC	Linear accelerometer (SI-compensated)	x, y, z
GYR	Gyroscope	x, y, z
MAG	Magnetometer	x, y, z
RAWGYR	Uncalibrated gyroscope	x, y, z, bias_x, bias_y, bias_z
RAWMAG	Uncalibrated magnetometer	x, y, z, bias_x, bias_y, bias_z
ORI	Rotation-vector quaternion	w, x, y, z
PRS	Barometer	pressure (hPa)
GPS	GPS fix	lat, lon, alt

Table 2: Per-sensor column schema emitted by the logger. Any row whose column count does not match its sensor tag is dropped silently by the parser (`src/data/loader/parsing.py`).

Once the recording is over, the raw dump is committed to `rawData/<exp_name>/`, together with a hand-filled `metadata.txt` (experimenter, phone, location, description, date, time). The loader pipeline (§2.5) then materialises a CSV per sensor under `structuredData/data/<exp_name>/` and derives `gt.csv` — the ground truth used by every downstream algorithm.

2.3 Elevator Detection

Ground truth for this project is a labelling of the timeline into `up`, `down`, and `outside` intervals: “outside” means the phone is not on a moving elevator (waiting, walking, idle). The labelling is derived algorithmically from the barometer signal and then optionally corrected by hand (§2.6).

2.3.1 Barometer-Based Algorithm

The detector lives in `src/algorithms/segmentation_algorithms/barometer_only/height_segmentation.py` and runs in six stages. Inputs: a time series (t_i, p_i) of timestamps and pressure readings.

1. **Altitude.** Convert pressure to altitude via the international barometric formula,

$$h_i = \frac{T_0}{L} \left[1 - (p_i/p_0)^{RL/(gM)} \right],$$

implemented in `src/physics/pressure_to_altitude.py`. This removes the weather-dependent DC offset only approximately, but the segmenter depends on *differences* in h , so the residual bias cancels.

2. **Low-pass smoothing.** Apply a centered rolling-mean filter of width `height_lowpass_sec · fs` samples to suppress high-frequency pressure jitter (door slams, HVAC bursts).
3. **Vertical velocity.** Estimate v_z by forward differencing the smoothed altitude, $v_{z,i} = (h_{i+1}^{\text{lp}} - h_i^{\text{lp}})/\Delta t_i$, then box-smooth again over `smooth_window_sec`.
4. **Thresholding.** Mark samples where $|v_z| > \text{velocity_threshold}$ (default ≈ 0.15 m/s) as “moving”. Extract contiguous runs of moving samples by edge-finding on the binary mask.
5. **Merging.** Walk the runs in order and merge consecutive runs of the *same sign* (both up or both down) if the gap between them is shorter than `merge_gap_sec`. This collapses spurious velocity dips caused by elevator deceleration mid-ride.
6. **Pruning and padding.** Drop any surviving run whose duration is $\leq \text{min_duration_sec}$ or whose total altitude change is $\leq \text{min_height_diff_m}$. Expand every surviving run by `pad_sec` on both ends so the label comfortably covers the full ride, including the small door-open tail.

The output is a `DataFrame` of rows (`start_ci`, `end_ci`, `duration`, `type`, `probability_ci`) where `type` $\in \{\text{up}, \text{down}\}$. The pipeline then pads the gaps between these rows with `outside` intervals so that the GT timeline is contiguous — this is what downstream evaluation code expects.

Why this is good enough as ground truth. A well-calibrated consumer barometer on a phone resolves pressure changes equivalent to roughly 10 cm of altitude at typical sampling rates, which is an order of magnitude below the smallest interesting inter-floor height (~ 3 m). The algorithm is deterministic, so repeated runs on the same recording produce identical labels, which in turn makes regression testing tractable.

2.4 The Time-Synchronisation Problem

The reason Section 2.3 alone does not close the loop is a low-level Android timing issue that we spent considerable time getting right.

The root cause. Android’s `SensorEvent.timestamp` is the time at which the sensor event was recorded, but it is expressed relative to the *device boot time* (`SystemClock.elapsedRealtimeNanos`), not to wall-clock. The logger writes this timestamp directly into the log file. Consequently, the `timestamp_ms` column in `sensorLog_*.txt` is a *device-local uptime* value, not a UTC time. Two phones that were booted at different moments will assign different timestamps to the *same* physical event, with the offset equal to the difference of their boot times — in our dataset, frequently tens of thousands of seconds, sometimes days.

Why this bites us in practice. Recall (§2.2) that phones without a barometer rely on a secondary device’s PRS stream for ground truth. To use the secondary device’s altitude as ground truth for the primary device, we must answer: *at primary timestamp t_p , what was the altitude?* That requires mapping primary time to secondary time, which requires knowing the boot-time offset between the two devices. Neither device records wall-clock, so the offset must be recovered from the data itself.

First approximation: start-time alignment. The simplest workable estimate is to assume both phones started recording at the same wall-clock moment, so the first sample of each log corresponds to the same real time. Under that assumption, the offset is

$$\Delta = t_0^{(\text{prim})} - t_0^{(\text{sec})},$$

and we shift every secondary timestamp by Δ before using it. This is implemented in `src/data/loader/alignment` and is sufficient when the experimenter hits “start” on both phones in quick succession.

Why start-time alignment is not enough. In practice the user latency between the two taps is one to several seconds and is variable between rides. That multi-second bias shifts the ground truth by enough to invalidate per-sample evaluation (an elevator moves at ~ 1.5 m/s; a 2 s timing error translates to ~ 3 m of altitude error, more than an entire floor). We therefore refine the offset using the **accelerometer signal**, which is recorded by both phones and must be approximately identical for two devices in the same hand.

Refinement by cross-sensor correlation. The routine used in the lab is: (i) compute the magnitude $|a|(t)$ on both phones, (ii) resample both to a common grid after applying the first-pass start-time offset, (iii) run a normalised cross-correlation of the two $|a|$ traces over a bounded lag window (± 10 s was enough in every experiment we observed), and (iv) pick the lag that maximises the correlation as the residual offset to fold back into Δ . If ACC is missing on either device, we fall back to gyroscope magnitude or heuristic event matching on step-count spikes. The diagnostic overlay (Figure 2) is produced automatically for every experiment with a `forBarometer/` companion and is the first thing to inspect when an alignment looks suspicious.

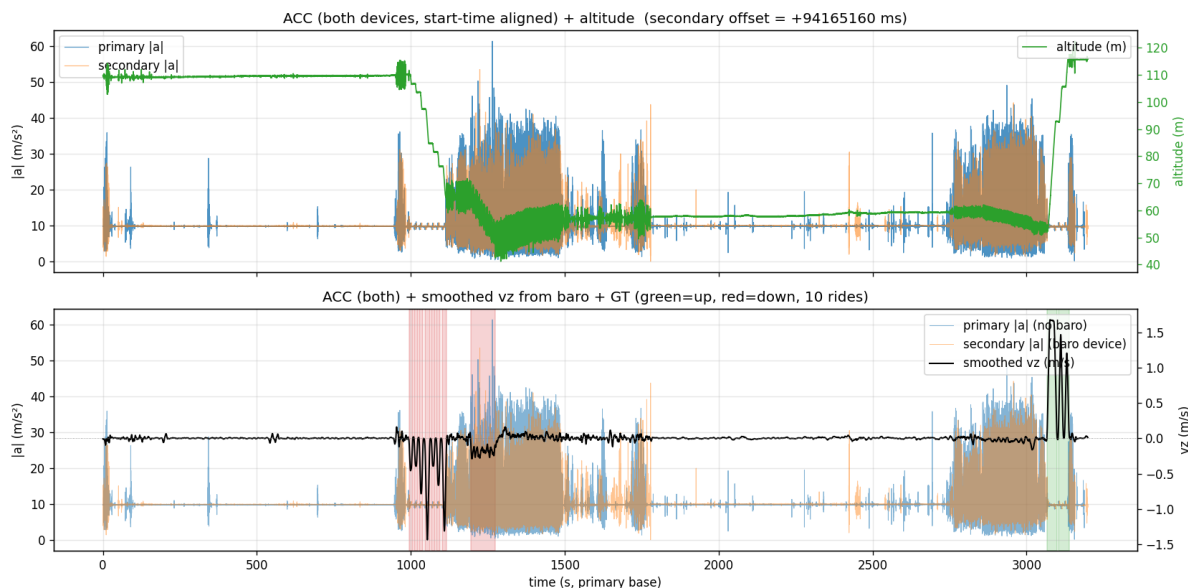


Figure 2: Diagnostic plot produced by `loader/alignment.py::_plot_forBarometer_alignment`.

Top: primary $|a|$ (blue) and time-shifted secondary $|a|$ (orange) overlaid, with the secondary’s altitude on the right y-axis (green). Signature features such as the impulse at door-close should line up after alignment. **Bottom:** the same accelerometer overlay with the barometer-derived smoothed vertical velocity in black and the detected ride intervals shaded (green = up, red = down). The reported offset is the value added to the secondary’s timestamps.

Practical hand-offs. The offset is computed once per experiment, logged, and baked into the secondary’s PRS frame before it is swapped into the primary’s sensor dictionary (`_merge_secondary_prs` in `alignment.py`). Everything downstream — the segmenter, the GT derivation, evaluation metrics — therefore sees *a single phone* with a valid barometer, which removes a whole class of timing bugs from the rest of the stack.

2.5 The Loader Pipeline

`src/data/loader/` exposes a narrow public API (`getExperimentData`, `saveExperimentData`, `list_experiments`) that hides all of the above. On first access it parses the raw `sensorLog_*.txt`, merges any `forBarometer/` secondary, runs the detector (§2.3.1), pads the timeline with outside intervals, and writes per-sensor CSVs plus `gt.csv` to `structuredData/data/<exp_name>/`. On every subsequent call it reads those CSVs directly. To force a rebuild after code changes, pass `use_cache=False`. The CSV schemas are documented in `src/data/README.md` and frozen in `loader/constants.py`.

2.6 The Ground-Truth Editor

No matter how careful the detector in §2.3.1 is, it will occasionally miss the start of a ride, split a single ride into two, or classify an HVAC-induced pressure ripple as movement. Rather than chase every edge case in the algorithm, we decided early on that the ground truth must be *auditable and editable* by a human. Every `gt.csv` in the dataset has therefore passed through the GT editor at least once.

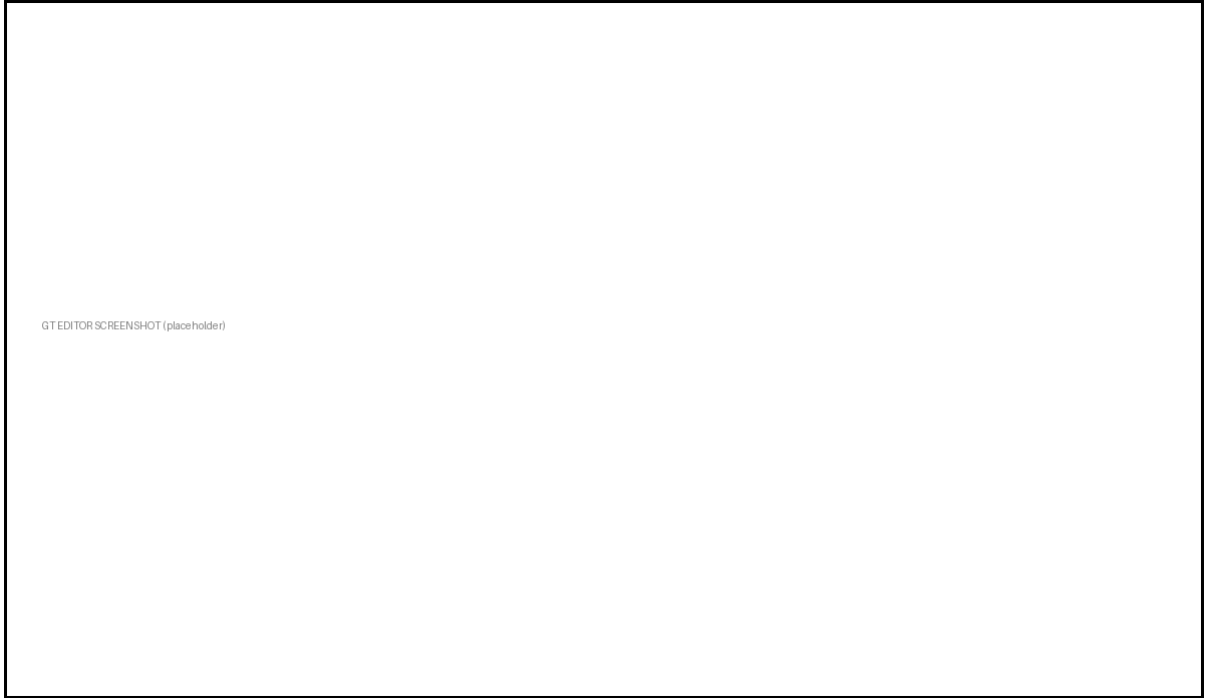


Figure 3: The GT editor. Left pane: stacked matplotlib panels (altitude, vertical velocity, $|a|$, $|\omega|$, $|m|$) with ground-truth intervals drawn as coloured spans (green = up, red = down, grey = outside). Right pane: a **Treeview** of all intervals with the edit form below it.

[TODO: drop the actual screenshot at docs/latex/figures/gt_editor_screenshot.png.]

Launching. The editor is a Tkinter application:

```
venv/bin/python -m src.data.gt_editor [exp_folder_name]
```

Called without arguments, it lists all experiments found under **rawData/** in a dropdown and lets the user pick one.

Visualisation. The left pane renders all five standard motion signals on a shared time axis. Ground-truth intervals are overlaid on every panel simultaneously so a user can sanity-check a label against multiple modalities at once (e.g. a true elevator ride must be accompanied by a clear altitude step *and* a flat-ish accelerometer magnitude).

Editing. The interaction model is designed to keep the mouse near the signal rather than in a form. Concretely:

- **Click** anywhere inside a coloured span to select that interval in the right-hand tree.
- **Drag an edge** (within a few pixels of the start or end of a span) to resize it live; the cursor morphs into a horizontal arrow while inside the hit-tolerance zone.
- Type precise millisecond bounds in the **edit form**, change the **type** (up / down / outside) or the free-text **note**, and hit **Apply**.
- + **Add** inserts a new 10-second **outside** interval right after the current selection.
- × **Delete** removes the current interval.

- **Auto-fix** closes gaps between adjacent intervals with **outside** filler so the timeline stays contiguous; it refuses to merge if two intervals actually overlap, to avoid silently destroying user edits.

Navigation. Keyboard shortcuts mirror the matplotlib toolbar: **+** / **=** zoom in, **-** zoom out, **0** fit to full range; **Shift+←** / **Shift+→** pan by 25% of the visible width; mouse-wheel over the plot zooms around the cursor. All of this is indispensable on long recordings where a single ride is only a few percent of the timeline.

Saving. **Ctrl+S** (or the **Save** button) calls `saveExperimentData`, which rewrites every per-sensor CSV, `gt.csv`, and `metadata.csv` under `structuredData/data/<name>/`, then rebuilds the top-level `metadata.csv` index. The floor-height lookup (`baramoshka.csv`) is intentionally *not* touched on save. The status bar at the bottom tracks unsaved-changes state; closing the window with pending edits prompts before quitting.



Figure 4: Fixing an auto-detection mistake in the GT editor. Here the barometer algorithm merged two adjacent rides into one because the cabin paused for less than `merge_gap_sec` at an intermediate floor; dragging the right edge of the first span and adding a new **down** interval splits the ride into its two true halves.

[TODO: drop an actual edit-example screenshot at docs/latex/figures/gt_editor_edit_example]

3 Segmentation & Detection

This section covers how a continuous accelerometer (and, when available, barometer) stream is sliced into elevator-ride segments. We begin with the underlying physics of an elevator ride — a four- parameter motion model that every detector in this work eventually reduces to — and then describe the concrete detectors built on top.

3.1 Background: Elevator Kinematics

Before describing any detector we lay down the physics of an elevator ride, because the same four parameters reappear in every algorithm we use later. Modern passenger elevators are commanded by a *jerk-limited motion profile* so that passengers experience a smooth, bounded change of acceleration (see, e.g., the ride-comfort specifications summarised in ISO 18738 and the manufacturer literature aggregated in `papers/elevator_research_links.txt`). The drive obeys, in order, the constraints

$$|\ddot{h}(t)| \leq j_{\max}, \quad |\dot{h}(t)| \leq a_{\max}, \quad |h(t)| \leq v_{\max}, \quad (1)$$

where $h(t)$ is the cabin altitude, $\dot{h} = v$ the vertical velocity, $\ddot{h} = a$ the acceleration measured by the phone (after gravity removal), and $\ddot{\dot{h}} = j$ the jerk. Typical commercial values are $j_{\max} \approx 1.0\text{--}1.6 \text{ m/s}^3$, $a_{\max} \approx 0.8\text{--}1.5 \text{ m/s}^2$, and $v_{\max} \approx 1.0\text{--}2.5 \text{ m/s}$ for low- and mid-rise buildings (CIBSE Guide D, 2020).

The seven-segment S-curve. Saturating each constraint in turn yields the canonical S-curve velocity profile (Figure 5). The ride decomposes into up to seven phases:

$$\underbrace{j = +j_{\max}}_{1: \text{ jerk up}} \quad \underbrace{j = 0}_{2: \text{ cruise } a_{\max}} \quad \underbrace{j = -j_{\max}}_{3: \text{ jerk down}} \quad \underbrace{a = 0}_{4: \text{ cruise } v_{\max}} \quad \underbrace{j = -j_{\max}}_{5: \text{ jerk down}} \quad \underbrace{j = 0}_{6: \text{ cruise } -a_{\max}} \quad \underbrace{j = +j_{\max}}_{7: \text{ jerk up}}.$$

Phase 4 collapses to zero length on short hops where the cabin never reaches $v_{\max}$, in which case the profile is “triangular” in acceleration; longer rides are “trapezoidal”. The constituent durations follow from (1):

$$T_j = \frac{a_{\max}}{j_{\max}}, \quad T_a = \max\left(0, \frac{v_{\max}}{a_{\max}} - T_j\right), \quad T_v = \max\left(0, \frac{H}{v_{\max}} - T_j - T_a\right), \quad (2)$$

where H is the floor-to-floor height. The total ride length is $T_{\text{ride}} = 4T_j + 2T_a + T_v$.

What the phone measures. The phone’s gravity-projected linear acceleration $a(t)$ traces *exactly* the bracketed acceleration in (1), modulo sensor noise and projection error. Three model facts are worth highlighting because the detector exploits them:

- During the four jerk phases (T_j each), $a(t)$ is linear in time with slope $\pm j_{\max}$. In the velocity domain this integrates to a quadratic — the parabolic “ramp shoulders” the user can see in Figures 5 and 6.
- The signed area under the first acceleration lobe equals the peak velocity reached:

$$v_{\text{peak}} = \int_0^{2T_j+T_a} a(t) dt = a_{\max} (T_j + T_a) = \min\left(v_{\max}, \sqrt{a_{\max} H}\right) \quad \text{for short rides.} \quad (3)$$

The second (deceleration) lobe has equal magnitude and opposite sign, so $\int_0^{T_{\text{ride}}} a(t) dt = 0$. This is the fundamental reason ZUPT works: the cabin starts and ends at rest, so any non-zero terminal velocity in the integrated signal is drift, not motion.

- The temporal gap between the centroids of the two acceleration lobes is the cruise time $T_v + 2T_a + 2T_j \approx H/v_{\max}$ when $v_{\max}$ is reached. **Hence the inter-pulse spacing is a direct, scale-free measurement of the floor-to-floor height H .**

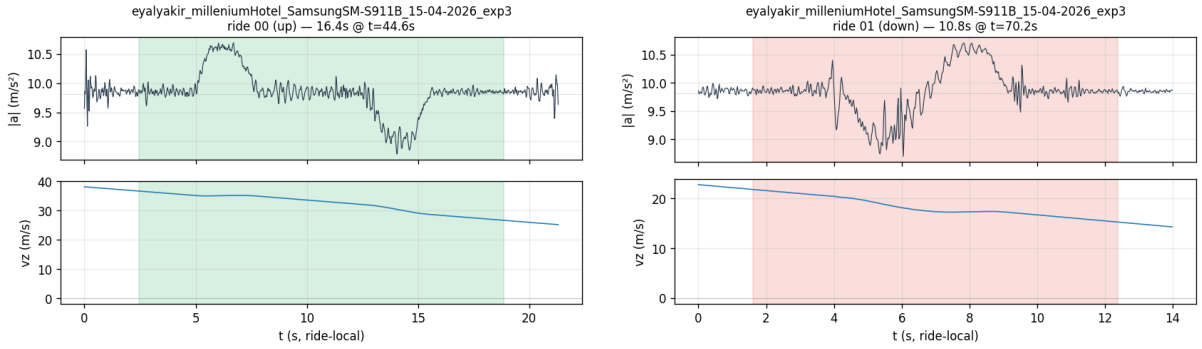
The four-parameter ride model. Combining (1)–(3), every ride is parameterised by the tuple

$$\theta_{\text{ride}} = (j_{\text{max}}, a_{\text{max}}, v_{\text{max}}, H), \quad (4)$$

with $j_{\text{max}}, a_{\text{max}}, v_{\text{max}}$ being properties of the elevator machine (constant within a building) and H being the only ride-specific quantity. Once θ_{ride} is fixed, the ideal acceleration template is fully determined:

$$a_{\theta}(t) = \begin{cases} +j_{\text{max}} t, & 0 \leq t < T_j, \\ +a_{\text{max}}, & T_j \leq t < T_j + T_a, \\ +a_{\text{max}} - j_{\text{max}}(t - T_j - T_a), & T_j + T_a \leq t < 2T_j + T_a, \\ 0, & 2T_j + T_a \leq t < 2T_j + T_a + T_v, \\ \text{(mirror of phases 1–3, with opposite sign),} & \text{otherwise.} \end{cases} \quad (5)$$

Velocity $v_{\theta}(t) = \int_0^t a_{\theta}(\tau) d\tau$ and displacement $h_{\theta}(t) = \int_0^t v_{\theta}(\tau) d\tau$ are then piecewise cubic and quartic respectively. Equation (5) is the synthetic template used by the DTW-based detector (Algorithm 2 in `papers/algorithm2_dtw.md`) and by the matched-filter pulse detector in `template_match/scripts/acc_pulse_detect.py`.



(a) Ride going *up*: positive accel lobe first, negative lobe at landing. The shaded interval is the GT label.

(b) Ride going *down*: lobes are mirrored. The vertical velocity panel makes the trapezoidal $v(t)$ profile clear.

Figure 5: Two single-ride zooms drawn from `eyalyakir_millenniumHotel_SamsungSM-S911B_15-04-2026_exp3`. Top of each subfigure: gravity-removed $|a|$. Bottom: the smoothed vertical velocity v_z recovered by integration. The two parabolic lobes in $|a|$ are the jerk-limited acceleration and deceleration phases of (5); the area under each lobe equals v_{peak} via (3), and the gap between lobe centroids measures the cruise time T_v , which is proportional to the inter-floor height H .

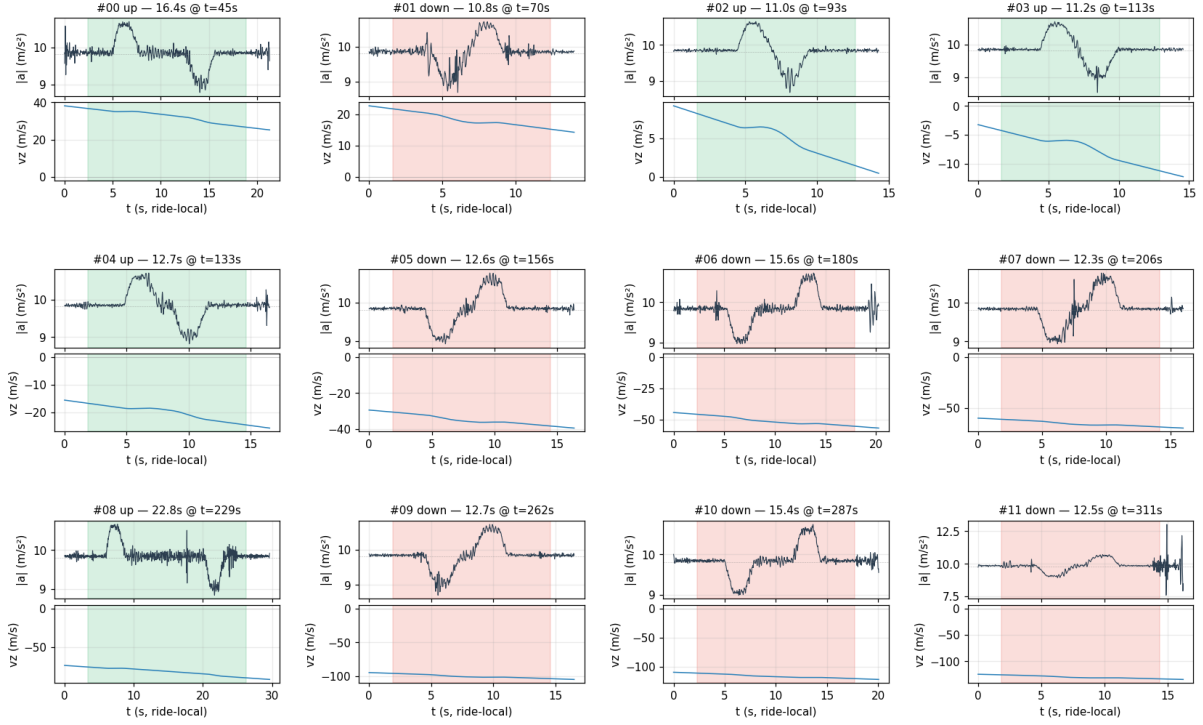


Figure 6: All twelve GT rides from the same experiment, on a common layout. The repeating two-lobe “ $+a$ then $-a$ ” structure predicted by (5) is unmistakable across rides of varying duration. Wider lobe spacing corresponds to larger H (more floors traversed), while the lobe *height* saturates at the building’s $a_{\max}$ regardless of H .

Implications for detection. The four-parameter model (4) gives every detector built on top concrete physical knobs:

- State-machine detectors (Algorithm 1 in `papers/algorithm1_state_machine.md`) threshold on $a_{\max}$ to find lobe entries and on a pulse-duration floor of $\sim T_j$.
- DTW-based template matching (Algorithm 2 in `papers/algorithm2_dtw.md`) generates one template per plausible θ_{ride} and warps it.
- Velocity-integral / ZUPT detectors (Algorithm 3 in `papers/algorithm3_integral_zupt.md`) check the integral identity $\int a \, dt = 0$ and the lower bound $\int |v| \, dt \gtrsim H_{\min}$.

The barometer-based GT detector described in §2.3.1 uses none of this — it works in altitude space directly — but the kinematic decomposition above is what makes its output a sensible reference signal for the accelerometer-based algorithms that follow.

3.2 Whole-Signal Trapezoid Detector

Our production accelerometer-only detector works directly on the idealised two-lobe model from Section 3.1: a ride is a signed acceleration pulse at take-off followed by an opposite-sign acceleration pulse at landing. The algorithm is split into two stages that mirror the two halves of the problem. A *detection* stage finds candidate lobes in the raw signal; a *pair-filter* stage decides which pairs of lobes constitute genuine rides. Eight tunable parameters — four in each stage — are bundled in a single configuration object, so a sweep can move them all in lockstep.

3.2.1 Detection: Per-Sample Best Template

Preprocess. The raw three-axis accelerometer (a_x, a_y, a_z) is projected onto the per-session gravity direction to obtain the signed vertical component a_{vert} , then smoothed with a centered rolling-mean filter of width `SMOOTH_SEC` = 0.4 s to obtain $a(t)$. Working in the smoothed signal is essential because the matched-filter sweep below would otherwise lock onto high-frequency noise.

Template shape and its link to the kinematics. We consider a grid of trapezoidal pulse templates $\tau_{W,f}(t)$ parameterised by a half-width $W \in \mathcal{W}$ and a flat fraction $f \in \mathcal{F}$,

$$\tau_{W,f}(t) = \begin{cases} 1 & |t| \leq fW, \\ \frac{W - |t|}{(1-f)W} & fW < |t| \leq W, \\ 0 & |t| > W, \end{cases} \quad (6)$$

which is a deliberately simplified version of the first acceleration lobe of (5): the flat plateau stands in for the a_{max} cruise, and the two linear ramps for the jerk-limited onset and offset. The identification with the kinematic parameters $(j_{\text{max}}, a_{\text{max}}, v_{\text{max}})$ is direct. The first lobe of (5) lasts from $t = 0$ to $t = 2T_j + T_a$, so its half-width is

$$W = T_j + \frac{1}{2}T_a = \frac{a_{\text{max}}}{2j_{\text{max}}} + \frac{1}{2} \max\left(0, \frac{v_{\text{max}}}{a_{\text{max}}} - T_j\right), \quad (7)$$

with $T_j = a_{\text{max}}/j_{\text{max}}$ fixed by the jerk limit. The flat fraction is the ratio of the plateau to the whole lobe,

$$f = \frac{T_a}{T_a + 2T_j} = \frac{\max(0, v_{\text{max}}/a_{\text{max}} - T_j)}{\max(0, v_{\text{max}}/a_{\text{max}} - T_j) + 2T_j}. \quad (8)$$

Two instructive corner cases of (7)–(8):

- A short hop that never reaches v_{max} has $T_a = 0$, so $W = T_j = a_{\text{max}}/j_{\text{max}}$ and $f = 0$ — a purely triangular pulse.
- A long ride that spends substantial time at v_{max} has $T_a \gg T_j$, so $W \approx T_a/2$ scales with H/v_{max} while $f \rightarrow 1 - 2T_j/T_a$ approaches one.

Concretely, for the commercial limits quoted in (1) ($j_{\text{max}} \approx 1.3 \text{ m/s}^3$, $a_{\text{max}} \approx 1.0 \text{ m/s}^2$, $v_{\text{max}} \approx 1.5 \text{ m/s}$) we have $T_j \approx 0.77 \text{ s}$, so W in this dataset lives essentially in the range $[T_j, T_j + T_v/2] \approx [0.8, 2.5] \text{ s}$ and f in roughly $[0, 0.7]$. The grid is chosen to comfortably bracket that: $W \in [\text{W_MIN_S}, \text{W_MAX_S}] = [0.4, 3.0] \text{ s}$ with 30 samples and $f \in [\text{F_MIN}, \text{F_MAX}] = [0.05, 0.80]$ with 15 samples — $30 \times 15 = 450$ templates total. Both axes start at a *positive* floor: W_{min} is a hair below the smallest physically plausible half-jerk-ramp T_j , and $f_{\text{min}} = 0.05$ excludes the degenerate exactly-triangular pulse whose derivative is discontinuous (the matched filter’s score for a pure triangular template is unstable under any smoothing).

Whole-signal sweep. For every (W, f) we run a normalised matched filter over the entire smoothed stream, yielding per sample i the least-squares amplitude and coefficient of determination

$$\hat{A}_{W,f}(i) = \frac{\langle a_{[i]}, \tau_{W,f} \rangle}{\|\tau_{W,f}\|^2}, \quad R_{W,f}^2(i) = 1 - \frac{\|a_{[i]} - \hat{A}_{W,f}(i) \tau_{W,f}\|^2}{\|a_{[i]}\|^2}, \quad (9)$$

where $a_{[i]}$ denotes the length- $2W$ signal window centred at sample i . We then collapse the grid to per-sample summaries:

$$R^2(i), \hat{A}(i) = \text{unsigned argmax across } (W, f), \quad (10)$$

$$R_+^2(i), \hat{A}_+(i) = \text{argmax restricted to } \hat{A}_{W,f}(i) > 0, \quad (11)$$

$$R_-^2(i), \hat{A}_-(i) = \text{argmax restricted to } \hat{A}_{W,f}(i) < 0. \quad (12)$$

The per-sign arrays are kept so that the diagnostic UI can show *why* one sign lost even when the unsigned argmax picked the other.

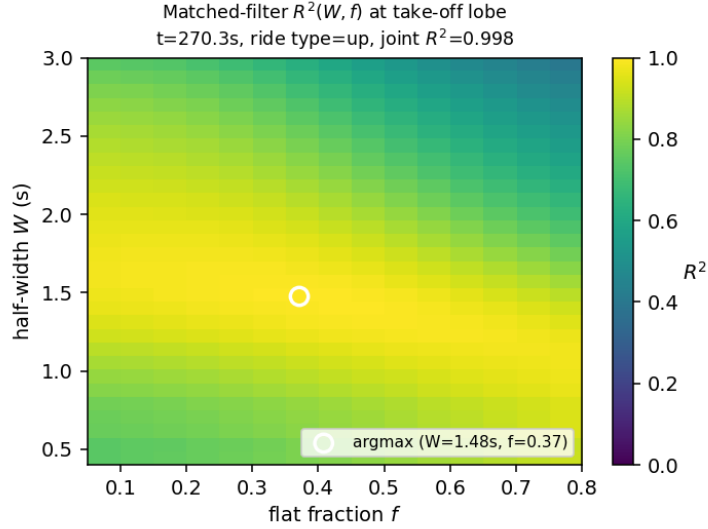


Figure 7: Matched-filter $R^2(W, f)$ evaluated at the sample corresponding to the take-off lobe of one accepted ride (highest-score pair in `eyalyakir_millenniumHotel_SamsungSM-S911B_15-04-2026_exp3`). The bright ridge on the grid is where the template shape agrees with the physical pulse; the marked argmax is the (W^*, f^*) the per-sample reduction would pick here. The ridge’s orientation is diagnostic: widening W at fixed f trades lobe sharpness for integration window, so a clean elevator lobe produces a compact bright patch centred on its true W , while a walking bump smears horizontally across f with a much lower peak R^2 .

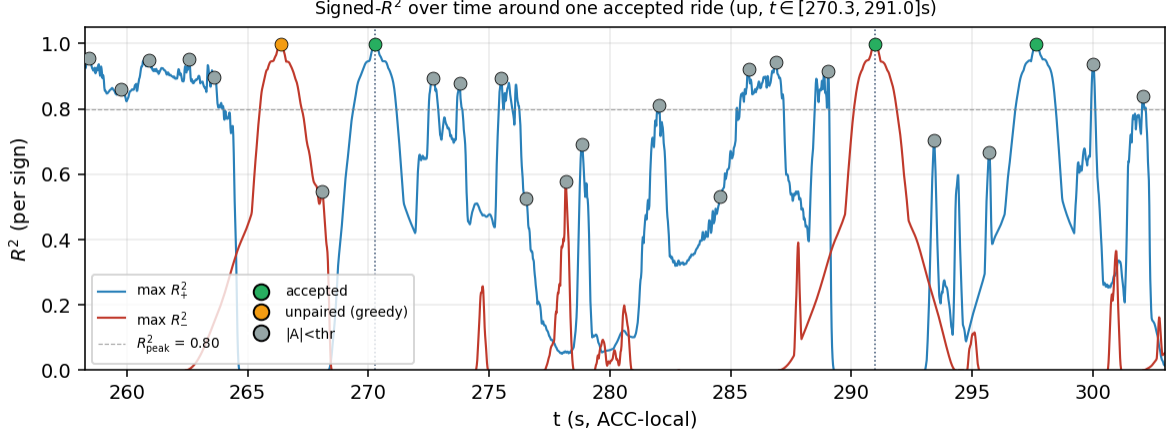


Figure 8: Per-sign R^2 traces $R^2_+(t)$ (blue) and $R^2_-(t)$ (red) on a twelve-second pad around the same accepted ride. The dashed horizontal line is `r2_peak_thresh`; vertical dotted lines mark the committed interval endpoints. Scatter dots are the local maxima of the signed traces, coloured by the first pipeline stage that kept or dropped each peak: green = **accepted** (survived every stage and is one of the two committed lobes), orange = **unpaired (greedy)** (made `final_peaks` but no pair involving it cleared `joint_r2_thresh`), purple = **same-sign NMS / NMS (local)** (suppressed by non-maximum-suppression), slate = **lost to opp sign** (the signed R^2 was high, but the unsigned argmax elected the opposite sign), and the grey shades are the below-threshold drops. This is the same colour legend the live editor UI renders; it is the primary debugging view when a GT ride is missed.

Peak-pick. Candidates are strict interior local maxima of $R^2(i)$ gated by the amplitude floor $|\hat{A}(i)| \geq \text{min_peak_abs_a}$ and the R^2 threshold $R^2(i) \geq \text{r2_peak_thresh}$, followed by a small-radius non-maximum suppression of `nms_radius_s` (default 0.5 s). This first NMS kills duplicate peaks that a single physical pulse produces at neighbouring samples.

Same-sign NMS. A second, larger NMS is applied *within each sign separately*: two candidates carrying the same sign $s_i = \text{sign } \hat{A}(i)$ must be at least `same_sign_min_gap_s` apart (default 20 s); the higher- R^2 candidate wins its window. Real elevator physics does not place two take-offs or two landings that close together, so this pass removes the secondary lobes the matched filter sometimes picks up around a true pulse.

The output of stage 1–4 is `final_peaks` — a list of sample indices $\{i_k\}$, each tagged with a sign $s_k \in \{+1, -1\}$ from $\text{sign } \hat{A}(i_k)$.

3.2.2 What Goes Wrong with Raw Peaks

Running just stages 1–4 of Section 3.2.1 — per-sample sweep plus two NMS passes — produces a set of *candidate lobes*, not rides. On our dataset this raw candidate list fails in three characteristic ways that we could see clearly in the editor’s signed- R^2 panel (Figure 8):

1. **False positives on pedestrian motion.** A brisk step sequence, a bag drop, or a door slam produces single-lobe accelerometer bursts that clear both the R^2 and amplitude thresholds. In isolation they look identical to an elevator take-off. In Figure 8 these are the orange / slate dots outside the committed interval.
2. **Mis-paired lobes.** Two unrelated high- R^2 peaks with opposite signs can sit within the admissibility window, and on template-per-peak arguments they would be happily

paired. But their individual best templates $(W_+^*, f_+^*) \neq (W_-^*, f_-^*)$ disagree —the “take-off” lobe was actually a footstep with $W \approx 0.5$ s and the “landing” lobe was an elevator deceleration with $W \approx 1.5$ s. A real ride has a single cabin producing both lobes with the *same* underlying shape.

3. **Swallowed short rides.** A “big” candidate pair stretching over a long pause between two physically distinct short rides can out-score the two correct short-ride pairs if we rank pairs only by mean R^2 . The big pair claims the whole time window, making the two real rides look like false positives on its shoulders.

Raising the per-peak thresholds does not fix (1): the false-positive peaks are not low- R^2 , they are structurally identical to real lobes until we look at their *partner*. Raising the pair-gap bounds does not fix (2): the gap itself is fine; it is the shape agreement that is off. And ranking pairs differently does not fix (3) unless we make the committed set mutually compatible.

The clearing algorithm in Section 3.2.3 addresses all three at once: (1) a lone candidate without an opposite-sign partner never survives — the filter is defined on pairs, not lobes; (2) the *shared-shape* joint fit below collapses the two lobes onto a single (W, f) template before scoring, so mismatched pairs score poorly and are rejected; (3) the greedy resolver enforces interval non-intersection, so two short-ride pairs with the same total score as one overlapping long pair always win as a pair.

3.2.3 Pair Filter: Lobe Pairing and Conflict Resolution

A single lobe is not yet a ride. The clearing algorithm walks every admissible (take-off, landing) pair, refits a *joint* template with a shared shape and shared amplitude, and then greedily commits a non-overlapping subset.

Admissibility. For a pair of candidates $(i_1, s_1 = +1)$ and $(i_2, s_2 = -1)$ — or the mirror $(-1, +1)$ for a downward ride — with $i_1 < i_2$, we require the temporal gap

$$\text{min_ride_s} \leq t(i_2) - t(i_1) \leq \text{max_ride_s},$$

where the upper bound (2 min by default) is a hard ceiling on realistic ride durations and the lower bound rejects a take-off misread as its own landing.

Shared-shape joint fit. For each template $\tau_{W,f}$ on the grid we centre one copy at i_1 and one at i_2 with the signs s_1, s_2 fixed externally, and fit a *single* amplitude $A \geq 0$ to both lobes jointly. Writing $a_k = a_{[i_k]}$, $P_k = \|a_k\|^2$, $\tau = \tau_{W,f}$, $N = \|\tau\|^2$, and $u_k = s_k \langle a_k, \tau \rangle$, the least-squares solution is closed form:

$$A^*(W, f) = \frac{u_1 + u_2}{2N}, \quad R_k^2 = 1 - \frac{P_k - 2A^*s_k \langle a_k, \tau \rangle + (A^*)^2 N}{P_k}, \quad (13)$$

and we score the pair by the mean of the two per-lobe R^2 ,

$$\text{score}(W, f) = \frac{1}{2}(R_1^2 + R_2^2). \quad (14)$$

The pair’s final score is the grid-max of (14), with (W^*, f^*) the template whose *same* shape best explains both lobes simultaneously. This is what distinguishes the pair filter from two independent peak fits: a true ride has two lobes of the same shape — same cabin, same drive — and the shared-shape constraint suppresses pairs whose lobes happen to clear the per-lobe R^2 threshold but with incompatible templates (e.g. two unrelated walking spikes).

Pair-level acceptance. A pair is accepted iff

$$\text{score}(W^*, f^*) \geq \text{joint_r2_thresh} \quad \text{and} \quad A^* \geq \text{min_pair_abs_a}.$$

Both thresholds are genuinely pair-level: a pair whose individual lobes passed the single-peak thresholds of Section 3.2.1 can still be rejected here if the lobes refuse to agree on a common template.

Greedy conflict resolution. We sort the accepted pairs by score in descending order and walk the list, committing a pair only if

1. neither of its lobes is already committed to another pair, *and*
2. its time interval $[t(i_1), t(i_2)]$ does not intersect any already-committed interval.

The second condition is the subtle one. Because a lobe can in principle participate in pairs with different partners at different gaps, simply enforcing “no shared lobe” still admits a “big” pair (covering many short rides) alongside the short rides it swallows. The interval non-intersection constraint gives the highest-scoring partition a veto over its territory, and in the dataset this almost always resolves in favour of the higher- R^2 short-ride fits, as expected.

3.2.4 Segment Endpoints

For each committed pair the output interval is simply

$$[t_{\text{start}}, t_{\text{end}}] = [t(i_1), t(i_2)],$$

i.e. the start and end of the ride are taken directly from the sample times of the two committed lobe *centres*, not from ramp-edge crossings or velocity thresholds. This is the natural choice given the kinematic model of Section 3.1: i_1 is (approximately) the centroid of the first acceleration lobe and i_2 the centroid of the second, so the gap $t(i_2) - t(i_1)$ is precisely the inter-pulse spacing whose proportionality to H/v_{max} is given by (2). The ride type is read off from the first-lobe sign: `ride_type` = `up` if $s_1 = +1$, else `down`.

A useful consequence is that the predicted interval is *contained* within the physical ride: it covers the cruise and the two cruise shoulders but not the brief door-open tail that the barometer-based GT labels (Section 2.3.1) intentionally pad. The evaluator therefore scores overlap fractionally — a GT and prediction “match” if either their absolute overlap is at least a second or their overlap covers at least 30% of the shorter interval — so that a correct-but-tight prediction is not penalised for missing the padding.

3.2.5 Tunables and How We Calibrated Them

Eight scalar thresholds control the whole detector. Four live in the detection stage and four in the pair filter. Every one of them has a direct physical interpretation in the kinematic model of Section 3.1, and every one was tuned empirically via a grid sweep on the 22 training experiments. The chosen values, their physical meaning, and the failure mode they guard against are summarised in Table 3; the methodology that produced those values is described below.

Parameter	Value	Physical meaning and failure mode guarded against
<i>Detection stage (§3.2.1)</i>		
<code>r2_peak_thresh</code>	0.75	Minimum shape-agreement of a candidate with its best trapezoid template. Below this, the window isn't "trapezoidal enough" to treat as a lobe. Guards against thresholding on low- R^2 noise.
<code>min_peak_abs_a</code>	0.5 m/s ²	Minimum LS-fit $ \hat{A} $ for a peak. Pedestrian bumps are the <i>right shape</i> (small trapezoids) but the <i>wrong size</i> ; this floor separates them from the building's $a_{\max}$ -scale cruise lobes. This is the principal pedestrian-FP lever.
<code>nms_radius_s</code>	0.5 s	Duplicate-peak dedup radius. A single physical pulse produces adjacent high- R^2 samples; this collapses them to one. Fixed by the sampling rate (~ 100 Hz) and cruise-onset duration T_j .
<code>same_sign_min_gap_s</code>	5.0 s	Minimum gap between two same-sign candidates. Real elevator physics doesn't place two take-offs (or two landings) closer than the shortest plausible ride, so a low value here lets back-to-back short rides survive while still killing in-pulse duplicates.
<i>Pair-filter stage (§3.2.3)</i>		
<code>min_ride_s</code>	10 s	Lower bound on the take-off→landing gap of a pair. Shorter gaps would be "dwell pairs" — the landing of one ride matched against the take-off of the next.
<code>max_ride_s</code>	120 s	Upper bound on the pair gap; a hard ceiling on ride duration.
<code>joint_r2_thresh</code>	0.75	Minimum shared-shape joint R^2 for an accepted pair. This is the pair-level analogue of <code>r2_peak_thresh</code> but harder to clear: a pair's two lobes must agree on the <i>same</i> (W, f) template, so mismatched lobes (foot-step take-off plus lift landing) fail here even if each lobe individually cleared the peak-level R^2 .
<code>min_pair_abs_a</code>	0.3 m/s ²	Minimum shared $ A $ of the accepted pair. Acts as a secondary amplitude floor at pair level. Lower than the peak-level floor because joint fitting already implicitly averages the two lobes.

Table 3: The detector's eight thresholds and the values we chose for them. Each threshold targets a distinct failure mode on the raw accelerometer signal; the values were picked by grid search (see below).

How the values were picked. We ran a grid search over the six non-kinematic knobs while pinning `min_ride_s` = 10 s (shortest plausible floor-to-floor ride) and `max_ride_s` = 120 s (upper bound on realistic rides). The grid had $2 \times 4 \times 1 \times 3 \times 2 \times 3 = 144$ combinations. Each combination was scored on all 22 training experiments (415 GT rides total) with the composite metric introduced in Section 3.2.7. The winning configuration shown in Table 3 emerged at the top of that table and also produced the highest classical F1 @ IoU ≥ 0.5 on the pooled predictions. Lower thresholds than those listed tended to add false positives faster than recall; higher thresholds tipped the failure mix into misses. The picked values are at the "recall cliff" on each axis.

3.2.6 Calibrating the Pair Filter by Iteration

Choosing the eight threshold values addresses only half of the problem. The structure of the pair filter itself — specifically, the rule by which the greedy conflict resolver ranks candidate pairs — matters at least as much, and cannot be tuned by a scalar sweep. We therefore ran a second round of calibration in which the detection-stage thresholds were frozen at the values of Table 3 and *only the pair-filter algorithm* changed. Ten variants were evaluated against the same 22-experiment set, each with a single targeted change versus the baseline.

What the baseline looked like. The baseline filter ranked accepted pairs by their mean joint R^2 and applied the greedy conflict resolver described in Section 3.2.3. On its own, that rule produced a specific pathology: take-offs from one ride and landings from a *different* ride several rides later tended to score high joint R^2 simply because both lobes were genuine lift lobes (same building, same cabin, same shape). The resulting “super pair” covered every GT ride in between and, by the interval-non-intersection rule, blocked the true short-ride pairs from committing. The failure mode dominated the error mix: 208 of 415 GT rides were swallowed by a merged prediction. Figure 9 shows the effect visually — the long purple blocks of predictions cover many green/red GT intervals each.

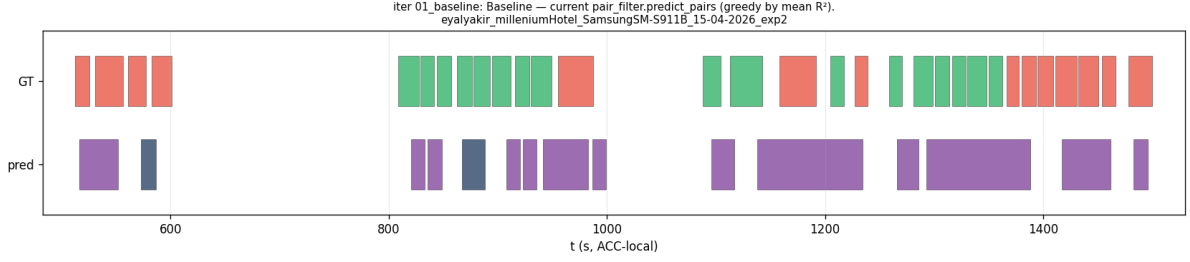


Figure 9: Baseline pair-filter output on the worst-merge experiment. Top row: GT rides (green = up, red = down). Bottom row: accepted predictions (blue = up, purple = down). Long purple bars span five or more consecutive GT rides each — the super-pair failure mode. 50 % of all GT rides in the dataset land under a merged prediction like one of these.

Variants tried. Each of the ten iterations swapped in a different acceptance or ranking rule and kept everything else identical:

1. **Baseline** — mean- R^2 greedy ranking.
2. **Duration penalty, light.** Rank by score $-\lambda \cdot \Delta t$ with $\lambda = 0.001$, so that at similar scores the shorter pair wins.
3. **Duration penalty, medium** ($\lambda = 0.003$).
4. **Duration penalty, heavy** ($\lambda = 0.01$). A 100 s pair now has to beat a 10 s pair by $\geq 0.9 R^2$ to outrank it.
5. **Mutual nearest neighbour.** Accept a pair only if each lobe is the other’s nearest admissible opposite-sign peak.
6. **Quiet middle.** Reject a pair if the signal between the two lobes is noisier than $0.5 A^*$ at any point.
7. **Min- R^2 acceptance.** Require $\min(R_1^2, R_2^2) \geq \text{joint_r2_thresh}$ (rather than the mean).
8. **Mutual nearest + duration penalty (medium).**
9. **Quiet middle + min- R^2 .**
10. **Kitchen sink.** All four above, stacked.

The score progression across the ten iterations is plotted in Figure 10.

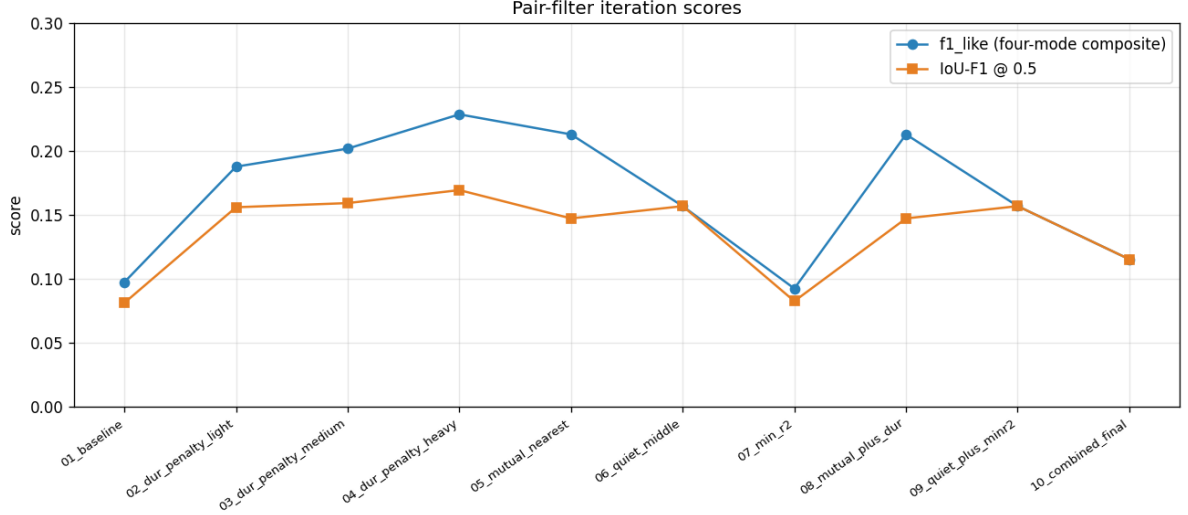


Figure 10: Scores across the ten pair-filter variants. Blue: the four-mode composite of Section 3.2.7. Orange: classical F1 at $\text{IoU} \geq 0.5$. The duration-penalty family (iterations 2–4) monotonically improves both scores up to $\lambda = 0.01$, at which point the score more than doubles versus baseline. Stacking rules (iter 10) over-regularises the filter and costs recall.

What worked: duration penalty. The biggest gain came from the simplest change: subtracting $0.01 \cdot \Delta t$ from a pair’s score before ranking. Intuitively, super pairs and genuine long rides both score the same joint R^2 , but super pairs are arbitrarily long while genuine rides cluster around ten to thirty seconds; a linear cost in Δt collapses that asymmetry. Empirically the composite score went from 0.097 at baseline to 0.228 at $\lambda = 0.01$, the number of merged GTs halved, and the number of clean one-to-one matches almost tripled. Figure 11 shows the same experiment as Figure 9 after the fix: the super pairs are gone and predictions are of the correct length scale.

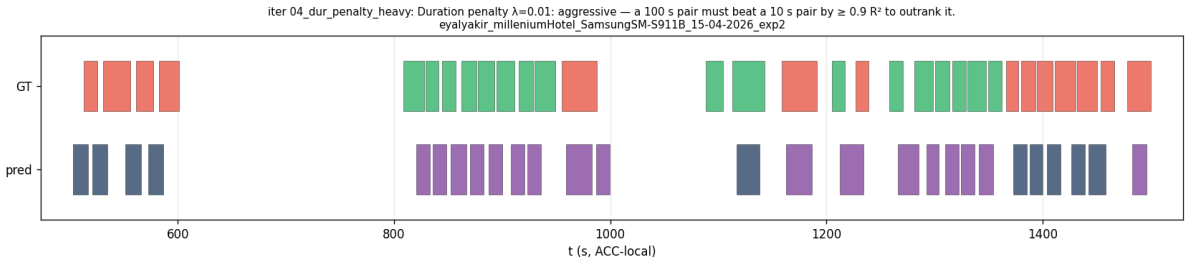


Figure 11: After adopting the duration-penalty rule ($\lambda = 0.01$) the super pairs of Figure 9 are gone. Predictions are now individual ~ 10 s bars. A residual failure mode is visible — some bars are offset from their matching GTs, because duration penalty monotonically prefers shorter gaps, so the landing of one ride occasionally gets paired with the take-off of the next (a “dwell pair”). A band penalty centred on typical ride length, or a higher `min_ride_s` floor, would close that remaining gap.

What didn’t work. The three “structural” variants (mutual nearest, quiet middle, $\text{min-}R^2$) each improved a single failure axis but hurt another more:

- **Mutual nearest neighbour** dropped false positives sharply — it refuses any pair where either lobe has a nearer admissible opposite — but missed more genuine rides whose lobes happen not to be each other’s nearest opposite.

- **Quiet middle** worked on paper but rejected many true rides whose cruise phase contained real signal (HVAC ripples, cable oscillations, cabin vibration). Recall collapsed.
- **Min- R^2** barely moved the needle. Mismatched pairs already had *both* lobes scoring well individually (each is a real lift lobe, just from a different ride); the mean-vs-min distinction doesn’t discriminate in that regime.

Stacking several of these constraints (iter 10) multiplied the recall hits. The lesson: constraints that prune entire families of candidates are expensive, while a soft ranking bias like the duration penalty can tilt the greedy in the right direction without categorically rejecting anyone.

3.2.7 Evaluation Metrics

Reporting a single detection F1 on temporal intervals hides several distinct failure modes we genuinely want to distinguish. We therefore evaluate every configuration and every iteration with two complementary metric families.

Four-mode composite. Given a GT set and a prediction set, we compute a bipartite overlap graph. Two intervals count as “matching” if their absolute overlap is at least one second or their overlap covers at least 30% of the shorter interval. We then categorise every GT and every prediction:

- *Clean* — one-to-one matches (good).
- *Missed* — a GT with zero overlapping predictions.
- *GT-merged* — a GT that shares its single overlapping pred with at least one other GT; symmetrically *pred-merged* on the pred side (the failure mode of Figure 9).
- *GT-split* — a GT covered by two or more predictions, and the corresponding *pred-split-part* on the pred side.
- *False-positive* — a prediction with zero overlapping GTs (lands on “outside”).

The aggregate score is

$$F1^* = \frac{2 \cdot \text{clean}}{2 \cdot \text{clean} + \text{bad}_{\text{GT}} + \text{bad}_{\text{pred}}}, \quad (15)$$

where $\text{bad}_{\text{GT}} = \text{missed} + \text{GT-merged} + \text{GT-split}$ and $\text{bad}_{\text{pred}} = \text{fp} + \text{pred-merged} + \text{pred-split-part}$. A perfect detector scores 1. Every failure category counts equally once, which is what makes the metric discriminating: a merge-heavy configuration and a miss-heavy configuration can score the same F1 on a pooled IoU evaluation but score very differently on (15), which is what we want when tuning the pair filter.

Classical IoU-F1 @ 0.5. For comparability with external temporal-detection work (PASCAL VOC, COCO temporal tracks, ActivityNet) we also report the standard F1 at intersection-over-union ≥ 0.5 : greedily assign each GT its highest-IoU pred, count a match if $\text{IoU} \geq 0.5$, and compute F1 over the pooled TP/FP/FN counts. This second number is directly comparable to any off-the-shelf interval-detection baseline; it is the orange curve in Figure 10.

Why both. The composite score (15) tells us which failure mode dominates and guides algorithmic changes (it was what surfaced the super-pair problem); the IoU-F1 tells us whether the *edges* of the intervals are well-placed once a match exists, and how the detector compares to generic temporal baselines. On our dataset the two curves track each other across iterations, which is a sanity check that the composite isn’t gamed by quirks of the four-mode definition.